Proyecto #2

Objetivo general del proyecto

El objetivo general de este proyecto es practicar varias etapas del desarrollo de una aplicación de software, desde el análisis hasta la construcción una aplicación funcional. A través del proyecto, los estudiantes pondrán en práctica todas las habilidades desarrolladas en el curso.

Objetivos específicos del proyecto

Durante el desarrollo de este proyecto se buscará el desarrollo de las siguientes habilidades:

1. Implementar interfaces de usuario basadas en consola, que implementen métodos adecuados para la verificación de las entradas
2. Describir historias de usuario con un buen nivel de detalle
3. Diseñar e implementar pruebas automáticas usando JUnit
4. Evaluar y mejorar el diseño del proyecto #1 con base en requisitos de evolución.

Instrucciones generales

Para este proyecto usted puede (debe) empezar a trabajar sobre su entrega para el proyecto #1. Pueden hacerse cambios sobre el diseño original para acomodar mejor los requerimientos de este proyecto.

Los grupos deben volver a conformarse para el proyecto #2. Los grupos nuevos pueden utilizar como base cualquiera de los proyectos #1 en el que hayan trabajado sus integrantes.

Nuevos requerimientos

Para esta parte del proyecto se agregará la siguiente funcionalidad al sistema:

Tras una mala temporada de ventas, el administrador del Board Game Café ha decidido crear torneos de juegos para aumentar el flujo de clientes. El administrador podrá crear torneos en un día en la semana en específico. Habrá dos tipos de torneo: Amistoso y Competitivo. En los torneos amistosos no se cobrará ninguna tarifa de entrada y se premiará con un bono de descuento redimible en la siguiente compra. En los competitivos se deberá pagar una tarifa de entrada y habrá un premio en metálico coherente con el dinero recaudado en la inscripción.

- Cada torneo debe tener un juego y número de participantes previamente definido, así como si el torneo es amistoso y competitivo. El número puede ser superior al número máximo de jugadores que admite el juego, siempre que las copias disponibles en el inventario de préstamos lo permitan. El administrador es el encargado de organizar cada torneo. Él es quien tiene el poder de crear nuevos torneos y definir sus características. Puede haber más de un torneo simultáneamente en el café.
- Los clientes pueden inscribirse o desinscribirse en los torneos a placer. Para cada torneo en el catálogo un usuario puede inscribir (Tomar un cupo) a un máximo de 3 participantes. Al hacer la desinscripción deben eliminarse todos los cupos tomados.

- Cada torneo tiene un 20% (Redondeado hacia arriba) de los cupos reservados para los jugadores fanáticos de este juego. Estos son aquellos clientes que han marcado dicho juego como uno de sus juegos favoritos. Si estos cupos se agotan, los fanáticos tomarán cupos regulares.
- Los empleados pueden inscribirse a los torneos siempre y cuando no estén cubriendo un turno el día de la realización del mismo. Los torneos competitivos serán gratuitos para los empleados, pero no se les dará el premio en metálico.
- Nuevamente, no es necesario pensar en los métodos y procesos de pago, pues están completamente tercerizados. Pero cada usuario deberá poder guardar el premio del descuento en caso de ganar un torneo amistoso. Este será usado en la compra que desee el usuario, y no es acumulable con otros descuentos.

Sobre el uso de Inteligencia Artificial Generativa

En 2024, la Universidad publicó los lineamientos para uso de la IAG. En este documento se definieron los diferentes niveles de uso de IAG para los ejercicios académicos de la universidad. Desde la coordinación y el grupo de profesores del curso somos conscientes del crecimiento de uso de IAG como herramienta complementaria o incluso principal en el rol de programador. Es importante que como programadores aprendan a usar estas herramientas ética y responsablemente. Pero es aún más importante que desarrollen sus habilidades como programadores competentes y de entendimiento y resolución de problemas por medio del análisis de contexto y diseño de soluciones. De modo, que para el desarrollo de los proyectos del curso el uso de la misma se encuentra admitido bajo ciertas condiciones.

Para la entrega única del proyecto 2 el nivel admitido es el **Nivel 3:**

Nivel 3	Uso de IA para editar y refinar	La IA generativa puede usarse para mejorar la claridad o calidad de las ideas del estudiante. El estudiante debe explicar cómo usó la herramienta en un pie de página o en un anexo del trabajo.

Es entendible que puedan y quieran utilizar IAG para la generación de código tal como *getters, setters* y constructores. Es más, el mismo IDE de Eclipse les da esta opción, así que no hay problema en estos casos. Sin embargo, para las funcionalidades críticas del sistema **NO ESTÁ permitido** el uso directo de esta. Sí es válido utilizarla para refinar código ya creado, depuración de errores y creación/replicación de plantillas o datos mock-up. También es válido utilizarla para generar el código base de un *InputStream* o *Scanner* que le permita la interacción básica en consola, aunque este se puede encontrar en StackOverflow o en las diapositivas del curso. En caso de no estar seguro de si el uso que quieran hacer esté contemplado en estas excepciones, **consulten directamente con el profesor de su sección.** En caso de usar IAG en alguno de estos casos, deben documentar el uso de la misma. La documentación del uso de IAG, incluyendo la colección de prompts usados **debe incluirse como anexo en la documentación del proyecto.**

NOTA IMPORTANTE: Si se sospecha un uso **irresponsable** o **no documentado** de la IAG en alguna de las entregas y que incumpla con los niveles autorizados, el profesor puede citar a una sustentación extra grupal o individual en que el estudiante deberá explicar con detalle la funcionalidad de su código. Si dicha sustentación resulta no ser satisfactoria, **la nota del entregable para todo el grupo podrá verse afectada negativamente.**

Entrega única: documentación e implementación

Actividades

1. Escriba en un documento las historias de usuario que se implementarán en la aplicación, para los tres tipos de usuarios principales: el administrador, los empleados y los clientes. Recuerde incluir en cada historia de usuario información sobre cómo son las entradas y las salidas/resultados en cada una. Los clientes deben poder hacer un registro por su cuenta para inscribirse al sistema, los empleados deben ser registrados por el administrador.
2. Implemente un conjunto de pruebas automatizadas para su aplicación **utilizando el framework Junit**.
    a. Construya pruebas unitarias para las clases que implementan la lógica de la aplicación. No vale la pena que escriba pruebas para métodos muy sencillos (ej., getters y setters): concéntrese en tener buenas pruebas que cubran los métodos que realmente valgan la pena.
    b. Si van a hacer cambios al código existente, intenten escribir antes las pruebas que la implementación (aplique TDD Test-Driven-Development).
    c. Construya pruebas de integración a partir de las historias de usuario. Debería haber **al menos una prueba por cada uno de los requerimientos funcionales.**
    d. **No es necesario** que construya pruebas para la interfaz (consola).
3. Implemente interfaces basadas en consola para los tres tipos de usuarios principales del sistema.
    a. Para cada tipo de usuario debe haber un método `main` diferente, pero todo debe estar implementado dentro del mismo proyecto Eclipse.
    b. Todos los usuarios deben autenticarse para poder utilizar las funcionalidades de la aplicación.
    c. Las interfaces deben permitir realizar todas las actividades necesarias en la aplicación según el tipo de usuario y deben validar todas las entradas que se den.
    d. Estructure las clases de la interfaz para que sean lo más fáciles de construir y, eventualmente, de extender. **Procure evitar clases muy complejas con miles de líneas para validar las entradas**.
    e. Al correr estas aplicaciones, deberían cargarse automáticamente los datos de archivos predeterminados y deberían persistirse los cambios en esos mismos archivos (v.g., no le pida al usuario el nombre de los archivos para cargar o para salvar). Sin embargo, **no es necesario que los cambios sean concurrentes** por cada acción o petición que se haga, basta con que se actualice al terminar el programa.
4. **Actualice el documento de diseño** de su aplicación con los cambios que haya introducido para soportar los nuevos requerimientos y con la implementación de la interfaz.

Entrega

1. El proyecto debe entregarse en una carpeta dentro del repositorio GIT del grupo con el nombre "**Proyecto 2"**. Dentro de esta carpeta debe existir una carpeta con el nombre "**Entrega única**" donde deben quedar todos los elementos correspondientes a esta entrega, incluyendo tanto los archivos fuente de los diagramas como imágenes que se puedan leer con facilidad y el proyecto Eclipse con instrucciones para su ejecución.
2. Entregue un enlace al repositorio a través de Bloque Neón en la actividad designada como "**Proyecto 2 - Entrega Única**".